Operating Systems Lab Assignment: Thread-Safe Data Structures

Christian Clarke

March 27, 2026

1 Introduction

This report covers thread-safe data structures implemented in C++. Mutexes ensure safe concurrent access.

2 Exercise 1: Queue and Stack

```
#include <iostream>
#include <queue>
#include <stack>
#include <mutex>
#include <thread>
#include <vector>
#include <string>
#include <chrono>
#include <atomic>

// Thread-safe Queue class
template <typename T>
class ThreadSafeQueue {
private:
    std::queue<T> queue;
    std::mutex mtx;

public:
    ThreadSafeQueue() {}

    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(value);
    }

    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (queue.empty()) return false;
        value = queue.front();
        queue.pop();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.empty();
```

```
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return queue.size();
    }
};

// Thread-safe Stack class
template <typename T>
class ThreadSafeStack {
private:
    std::stack<T> stack;
    std::mutex mtx;

public:
    ThreadSafeStack() {}

    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        stack.push(value);
    }

    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (stack.empty()) return false;
        value = stack.top();
        stack.pop();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return stack.empty();
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return stack.size();
    }
};

// Problem 1: Producer-Consumer Simulation
void producerConsumerProblem() {
    ThreadSafeQueue<std::string> messageQueue;
    const int NUM_PRODUCERS = 3;
    const int NUM_CONSUMERS = 2;
    const int MESSAGES_PER_PRODUCER = 5;
    std::atomic<int> messages_produced(0);
    std::vector<std::thread> producers;
    std::vector<std::thread> consumers;

    auto producer = [&messageQueue, &messages_produced](int id) {
        for (int i = 0; i < MESSAGES_PER_PRODUCER; ++i) {
            std::string message = "Producer␣" + std::to_string(id) + "␣
                Message␣" + std::to_string(i);
            messageQueue.push(message);
            messages_produced++;
```

```
            std::cout << "Produced: " << message << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto consumer = [&messageQueue, &messages_produced](int id) {
        int consumed = 0;
        while (consumed < NUM_PRODUCERS * MESSAGES_PER_PRODUCER) {
            std::string message;
            if (messageQueue.pop(message)) {
                std::cout << "Consumer " << id << " processed: " <<
                    message << std::endl;
                consumed++;
            } else if (messages_produced == NUM_PRODUCERS *
                MESSAGES_PER_PRODUCER) {
                break;
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    for (int i = 0; i < NUM_PRODUCERS; ++i) {
        producers.emplace_back(producer, i);
    }

    for (int i = 0; i < NUM_CONSUMERS; ++i) {
        consumers.emplace_back(consumer, i);
    }

    for (auto& t : producers) t.join();
    for (auto& t : consumers) t.join();
}

// Problem 2: Undo-Redo System
void undoRedoProblem() {
    ThreadSafeStack<std::string> undoStack;
    ThreadSafeStack<std::string> redoStack;
    std::string currentText = "";

    auto editText = [&undoStack, &redoStack, &currentText](const std::
        string& newText) {
        undoStack.push(currentText);
        while (!redoStack.empty()) {
            std::string temp;
            redoStack.pop(temp);
        }
        currentText = newText;
        std::cout << "Text updated to: " << currentText << std::endl;
    };

    auto undo = [&undoStack, &redoStack, &currentText]() {
        std::string prevText;
        if (undoStack.pop(prevText)) {
            redoStack.push(currentText);
            currentText = prevText;
```

```
            std::cout << "Undo - Current text: " << currentText << std
                ::endl;
        }
    };

    auto redo = [&undoStack, &redoStack, &currentText]() {
        std::string nextText;
        if (redoStack.pop(nextText)) {
            undoStack.push(currentText);
            currentText = nextText;
            std::cout << "Redo - Current text: " << currentText << std
                ::endl;
        }
    };

    editText("Hello");
    editText("Hello World");
    editText("Hello Universe");
    undo();
    undo();
    redo();
    editText("Hello Galaxy");
}

int main() {
    std::cout << "Problem 1: Producer-Consumer Simulation\n";
    producerConsumerProblem();

    std::cout << "\nProblem 2: Undo-Redo System\n";
    undoRedoProblem();
    return 0;
}
```

Explanation: Mutexes and lock_guard protect shared data.
Analysis: Prevents race conditions in producer-consumer and undo-redo.

3 Exercise 3: Priority Queue

```
#include <iostream>
#include <queue>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>
#include <cstdlib>

template <typename T>
class ThreadSafePriorityQueue {
private:
    std::priority_queue<T> pq;
    std::mutex mtx;

public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        pq.push(value);
    }
```

```
● @iamcjclarke → ~/thread_safe_lab $ ./thread_safe_data_structures
  Problem 1: Producer-Consumer Simulation
  Produced: Producer 1 Message 0
  Produced: Producer 0 Message 0
  Consumer 1 processed: Producer 0 Message 0
  Produced: Consumer 0 processed: Producer 1 Message 0
  Producer 2 Message 0
  Produced: Producer 1 Message 1
  Produced: Producer 0 Message 1
  Produced: Producer 2 Message 1
  Consumer 1 processed: Producer 2 Message 0
  Consumer 0 processed: Producer 1 Message 1
  Produced: Producer 0 Message 2
  Produced: Producer 1 Message 2
  Produced: Producer 2 Message 2
  Consumer 1 processed: Producer 0 Message 1
  Consumer Produced: Producer 0 Message 3
  Produced: Producer 2 Message 3
  Produced: Producer 1 Message 3
  0 processed: Producer 2 Message 1
  Produced: Producer 0 Message 4
  Produced: Producer 2 Message 4
  Produced: Producer 1 Message 4
  Consumer 1 processed: Producer 0 Message 2
  Consumer 0 processed: Producer 1 Message 2
  Consumer 1 processed: Producer 2 Message 2
  Consumer 0 processed: Producer 0 Message 3
  Consumer 1 processed: Producer 2 Message 3
  Consumer 0 processed: Producer 1 Message 3
  Consumer 1 processed: Producer 0 Message 4
  Consumer 0 processed: Producer 2 Message 4
  Consumer 1 processed: Producer 1 Message 4

  Problem 2: Undo-Redo System
  Text updated to: Hello
  Text updated to: Hello World
  Text updated to: Hello Universe
  Undo - Current text: Hello World
  Undo - Current text: Hello
  Redo - Current text: Hello World
  Text updated to: Hello Galaxy
```

Figure 1: Exercise 1 Output

```
    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (pq.empty()) return false;
        value = pq.top();
        pq.pop();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return pq.empty();
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return pq.size();
    }
};

void priorityQueueTest() {
    ThreadSafePriorityQueue<int> pq;
    std::vector<std::thread> threads;
```

```
    const int NUM_THREADS = 4;

    auto pusher = [&pq](int id) {
        for (int i = 0; i < 5; ++i) {
            int priority = rand() % 100;
            pq.push(priority);
            std::cout << "Thread␣" << id << "␣pushed:␣" << priority <<
                "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto popper = [&pq]() {
        for (int i = 0; i < 20; ++i) {
            int value;
            if (pq.pop(value)) {
                std::cout << "Popped:␣" << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    for (int i = 0; i < NUM_THREADS; ++i) {
        threads.emplace_back(pusher, i);
    }
    threads.emplace_back(popper);

    for (auto& t : threads) t.join();
}

int main() {
    priorityQueueTest();
    return 0;
}
```

Explanation: Thread-safe priority queue with multiple threads.
Analysis: Mutex ensures correct ordering and safe access.

4 Exercise 4: Circular Buffer

```
#include <iostream>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <vector>
#include <chrono>
#include <cstdlib>

#define BUFFER_SIZE 5

class ThreadSafeCircularBuffer {
private:
    int buffer[BUFFER_SIZE];
    int in = 0, out = 0, count = 0;
    std::mutex mtx;
```

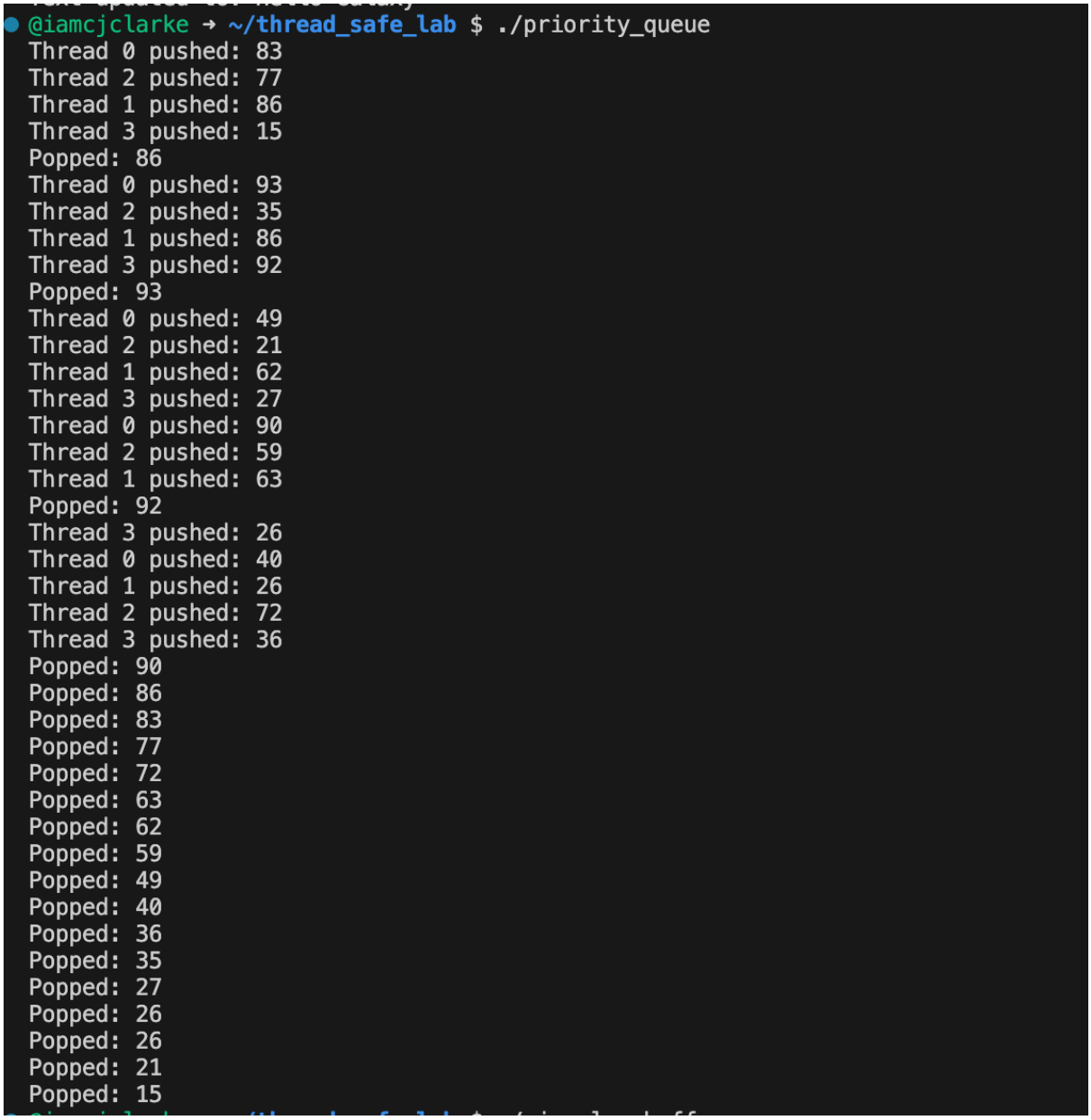

Figure 2: Exercise 3 Output

```
    std::condition_variable not_full, not_empty;

public:
    void push(int value) {
        std::unique_lock<std::mutex> lock(mtx);
        not_full.wait(lock, [this] { return count < BUFFER_SIZE; });
        buffer[in] = value;
        in = (in + 1) % BUFFER_SIZE;
        count++;
        lock.unlock();
        not_empty.notify_one();
    }

    bool pop(int& value) {
        std::unique_lock<std::mutex> lock(mtx);
```

```
        not_empty.wait(lock, [this] { return count > 0; });
        value = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        lock.unlock();
        not_full.notify_one();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return count == 0;
    }

    bool full() {
        std::lock_guard<std::mutex> lock(mtx);
        return count == BUFFER_SIZE;
    }
};

void circularBufferTest() {
    ThreadSafeCircularBuffer cb;
    std::vector<std::thread> producers, consumers;
    const int NUM_ITEMS = 10;

    auto producer = [&cb]() {
        for (int i = 0; i < NUM_ITEMS; ++i) {
            int value = rand() % 100;
            cb.push(value);
            std::cout << "Produced: " << value << "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto consumer = [&cb]() {
        for (int i = 0; i < NUM_ITEMS; ++i) {
            int value;
            cb.pop(value);
            std::cout << "Consumed: " << value << "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    producers.emplace_back(producer);
    consumers.emplace_back(consumer);

    for (auto& t : producers) t.join();
    for (auto& t : consumers) t.join();
}

int main() {
    circularBufferTest();
    return 0;
}
```

Explanation: Uses condition variables for synchronization.

Analysis: Prevents overflow/underflow and avoids busy waiting.

```
@iamcjclarke → ~/thread_safe_lab $ ./circular_buffer
Produced: Consumed: 8383

Produced: 86
Consumed: 86
Produced: 77
Consumed: 77
Produced: 15
Produced: 93
Consumed: 15
Produced: 35
Consumed: 93
Produced: 86
Produced: 92
Consumed: 35
Produced: 49
Consumed: 86
Produced: 21
Consumed: 92
Consumed: 49
Consumed: 21
```

Figure 3: Exercise 4 Output

5 Exercise 5: Deque

```
#include <iostream>
#include <deque>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>

template <typename T>
class ThreadSafeDeque {
private:
    std::deque<T> deque_;
    std::mutex mtx;

public:
    void push_front(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        deque_.push_front(value);
    }

    void push_back(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        deque_.push_back(value);
    }

    bool pop_front(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (deque_.empty()) return false;
        value = deque_.front();
        deque_.pop_front();
        return true;
    }
```

```
    bool pop_back(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (deque_.empty()) return false;
        value = deque_.back();
        deque_.pop_back();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return deque_.empty();
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return deque_.size();
    }
};

void dequeTest() {
    ThreadSafeDeque<int> dq;
    std::vector<std::thread> threads;

    auto push_front = [&dq](int id) {
        for (int i = 0; i < 5; ++i) {
            dq.push_front(id * 100 + i);
            std::cout << "Thread␣" << id << "␣pushed␣front:␣" << (id *
                100 + i) << "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto push_back = [&dq](int id) {
        for (int i = 0; i < 5; ++i) {
            dq.push_back(id * 100 + i);
            std::cout << "Thread␣" << id << "␣pushed␣back:␣" << (id *
                100 + i) << "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto pop_front = [&dq]() {
        for (int i = 0; i < 10; ++i) {
            int value;
            if (dq.pop_front(value)) {
                std::cout << "Popped␣front:␣" << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    auto pop_back = [&dq]() {
        for (int i = 0; i < 10; ++i) {
            int value;
```

```
            if (dq.pop_back(value)) {
                std::cout << "Popped back: " << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    threads.emplace_back(push_front, 1);
    threads.emplace_back(push_back, 2);
    threads.emplace_back(pop_front);
    threads.emplace_back(pop_back);

    for (auto& t : threads) t.join();
}

int main() {
    dequeTest();
    return 0;
}
```

Explanation: Supports thread-safe push/pop from both ends.
Analysis: Mutex ensures consistency when accessing both ends.

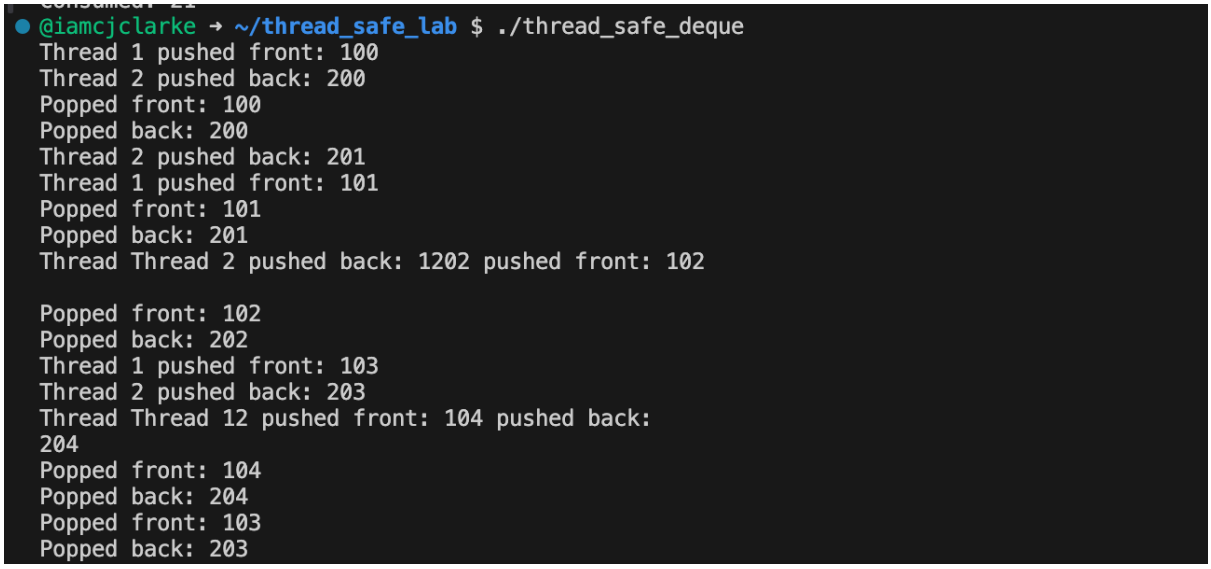

Figure 4: Exercise 5 Output

6 Exercise 6: Linked List

```
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>

template <typename T>
class ThreadSafeLinkedList {
private:
```

```
    struct Node {
        T data;
        Node* next;
        Node(T value) : data(value), next(nullptr) {}
    };

    Node* head = nullptr;
    std::mutex mtx;

public:
    ~ThreadSafeLinkedList() {
        while (!empty()) {
            T value;
            pop_front(value);
        }
    }

    void push_front(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        Node* newNode = new Node(value);
        newNode->next = head;
        head = newNode;
    }

    bool pop_front(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (!head) return false;
        Node* temp = head;
        value = temp->data;
        head = head->next;
        delete temp;
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return head == nullptr;
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        size_t count = 0;
        Node* current = head;
        while (current) {
            count++;
            current = current->next;
        }
        return count;
    }
};

void linkedListTest() {
    ThreadSafeLinkedList<int> list;
    std::vector<std::thread> threads;

    auto pusher = [&list](int id) {
        for (int i = 0; i < 5; ++i) {
            list.push_front(id * 100 + i);
```

```
            std::cout << "Thread␣" << id << "␣pushed:␣" << (id * 100 +
                i) << "\n";
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto popper = [&list]() {
        for (int i = 0; i < 10; ++i) {
            int value;
            if (list.pop_front(value)) {
                std::cout << "Popped:␣" << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    threads.emplace_back(pusher, 1);
    threads.emplace_back(pusher, 2);
    threads.emplace_back(popper);

    for (auto& t : threads) t.join();
}

int main() {
    linkedListTest();
    return 0;
}
```

Explanation: Thread-safe singly linked list implementation.
Analysis: Mutex protects pointer updates to maintain list integrity.

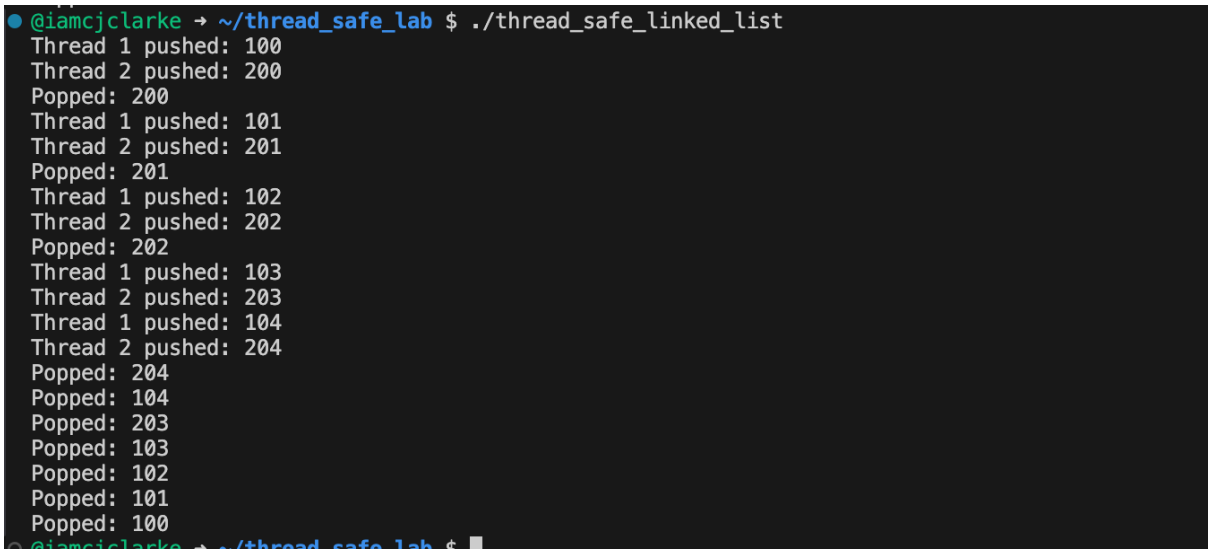

Figure 5: Exercise 6 Output